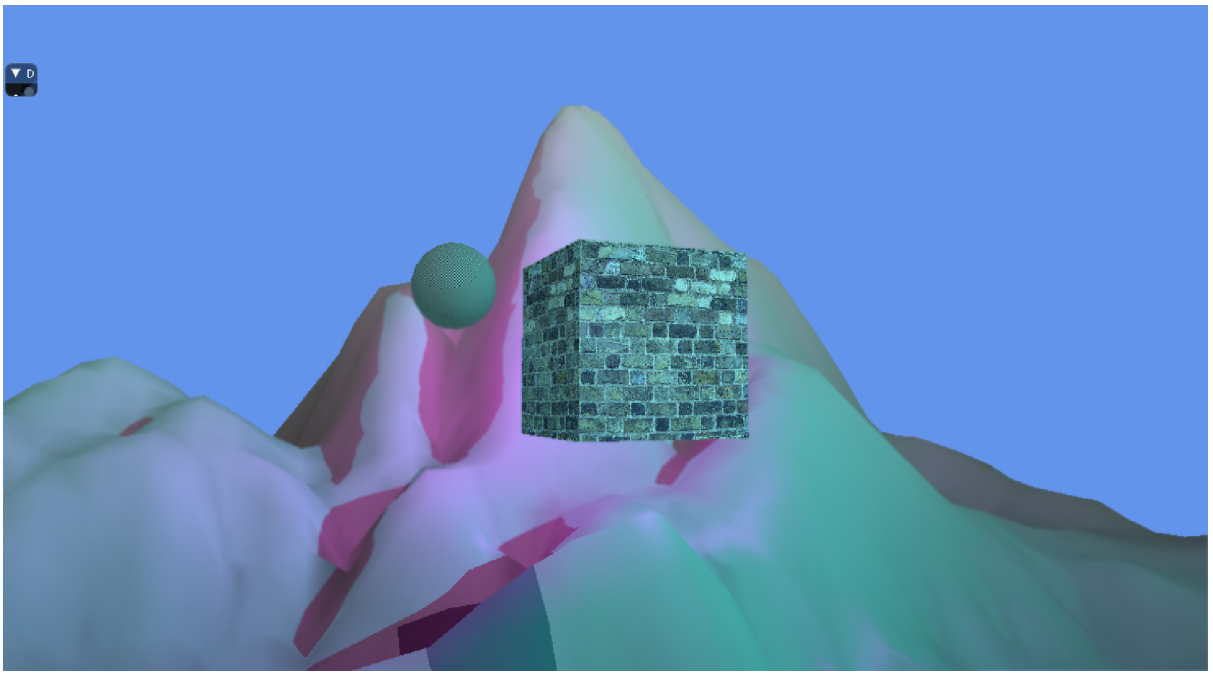
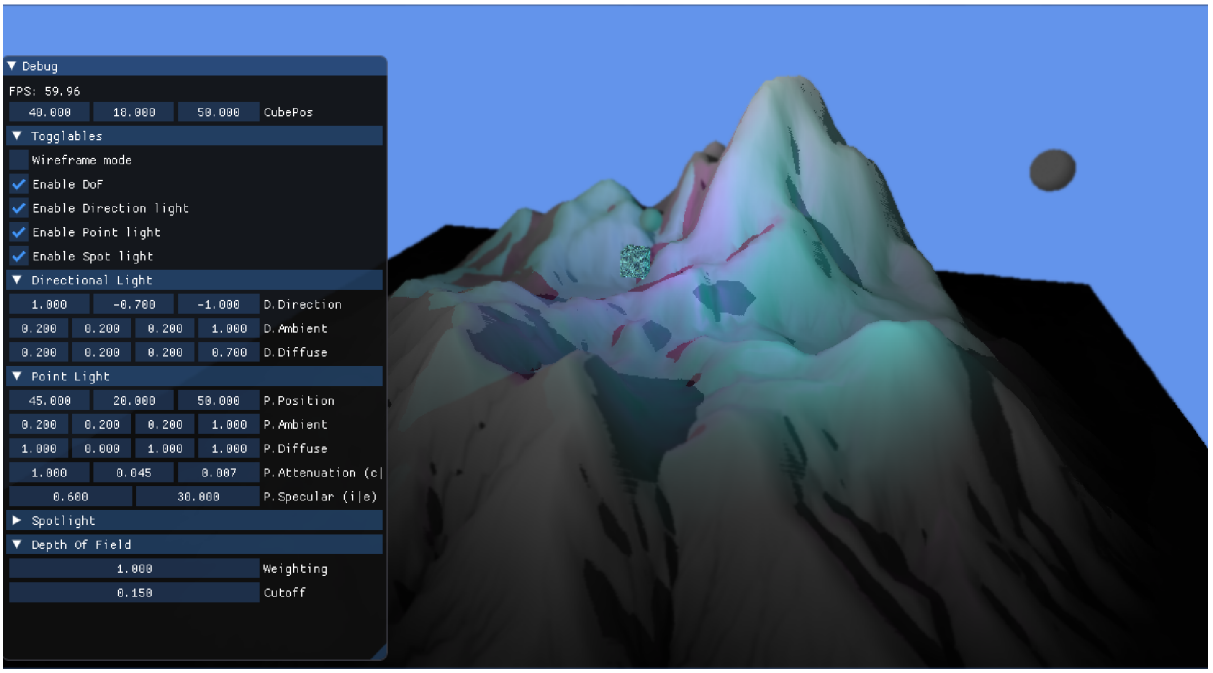
CMP301 Report

Overview

The scene I have created uses the provided height map to generate terrain, I have used vertex manipulation to turn the height map into a small mountain range. The most difficult part of this was getting the mountain range to be lit correctly, since the height map is a flat image all the normals were facing the same direction but obviously once it's exaggerated into a mountain range these normals are incorrect so calculations were needed to correct this. Within the scene there are 3 lights, a directional light which casts shadows onto the entire scene, a point light which while not casting shadows focused more on having specular attributes which can be seen along the mountain, and a spot light which also casts shadows similarly to the directional light, being a spotlight the user has a lot more control over exactly where and what the spot light covers as the cutoff angle can be changed to increase or decrease the spread of the light. The post processing effect that I chose was a depth of field post processing effect, this blurs the screen to simulate peripheral vision around the centre. The blur gradually increases in intensity as you get further from the centre of the screen.



As seen above the point light and spot light are coloured, this is all controllable in the GUI (which has been minimised in the screen shot). The different colours allow the user to clearly differentiate the lights, of course they can use the GUI to individually toggle any of the lights on or off. What's also visible in the screenshot is a cube, this cube was added to demonstrate shadows better. The cube is able to be moved using the sliders for its position in the GUI and as you move it, the shadow updates for both the spotlight and the directional light.



Finally the GUI shown above is used to control practically every element within the scene. Firstly it is formatted to be neat and easily understandable, each element with multiple sliders is grouped into its specific object, each light has its own drop down menu within which you can control each element of the light, the same is true for the depth of field post processing effect and there is a separate drop down menu for all toggles in the scene including turning lights on or off, enabling or disabling depth of field and enabling wireframe mode.

As previously mentioned, the GUI can be used to move the cubes position in the scene. All lights also have a controllable position within this GUI, all positions are controlled by 3 sliders to control the values for their X,Y, and Z coordinates respectively, this is the same for the cube's position. Similarly all lights have sliders similar to direction for their ambient and diffuse attributes, however since these are stored as float4 type variables internally there are 5 sliders to control the red green blue and alpha values that comprise these attributes. The directional and spot light also have controls for the direction the lights shine, and the spot light has an additional slider for the cutoff angle to change the focus of the spotlight. Previously I mentioned the point light having specular attributes there are also GUI sliders to control the specular values as well as the attenuation for that light specifically. Finally the GUI also adds controls for adjusting the depth of field post processing effect, being able to change the weighting to change the harshness of the blur and a cutoff to determine how much of the screen gets blurred.

Explanation

Vertex Manipulation

Outline
The mountain terrain in this scene is generated using a displacement map, or more specifically a height map. To do this a plane is generated, however this plane is completely flat, so in the manipulation vertex shader a height value is needed, to calculate this a function was made to sample the height map texture and equate the height to the colour's x value in the height map, these values wouldn't create satisfying mountains so this value is then multiplied by 30 to extend this even further in the vertex shader the y value for position of vertices is set to the calculated height value, then these new coordinates are set, this results in the mountain mesh that is seen. The issue with this is that the normals are now completely incorrect as the initial mesh calculated normals to all point directly up (+y), now the plane has been manipulated; these normal values stay the same and for most of the geometry this is now incorrect. so a separate calculation is required to calculate the new normals.

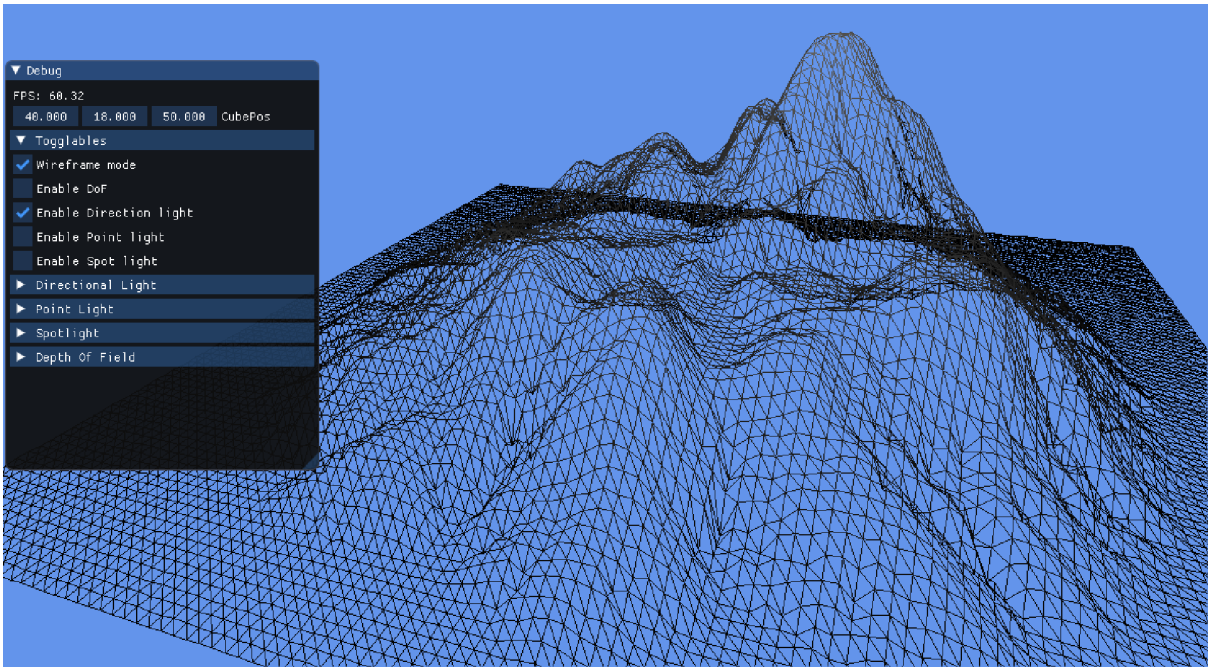
The vertices are now located correctly however none of their normals have been calculated, this would mean that lighting will be completely inaccurate. To calculate the normal for a given their neighbour vertices are used to get tangents which when the cross product is applied to them that will give the normal of the target vertex.

To further explain this process, it occurs over 4 stages

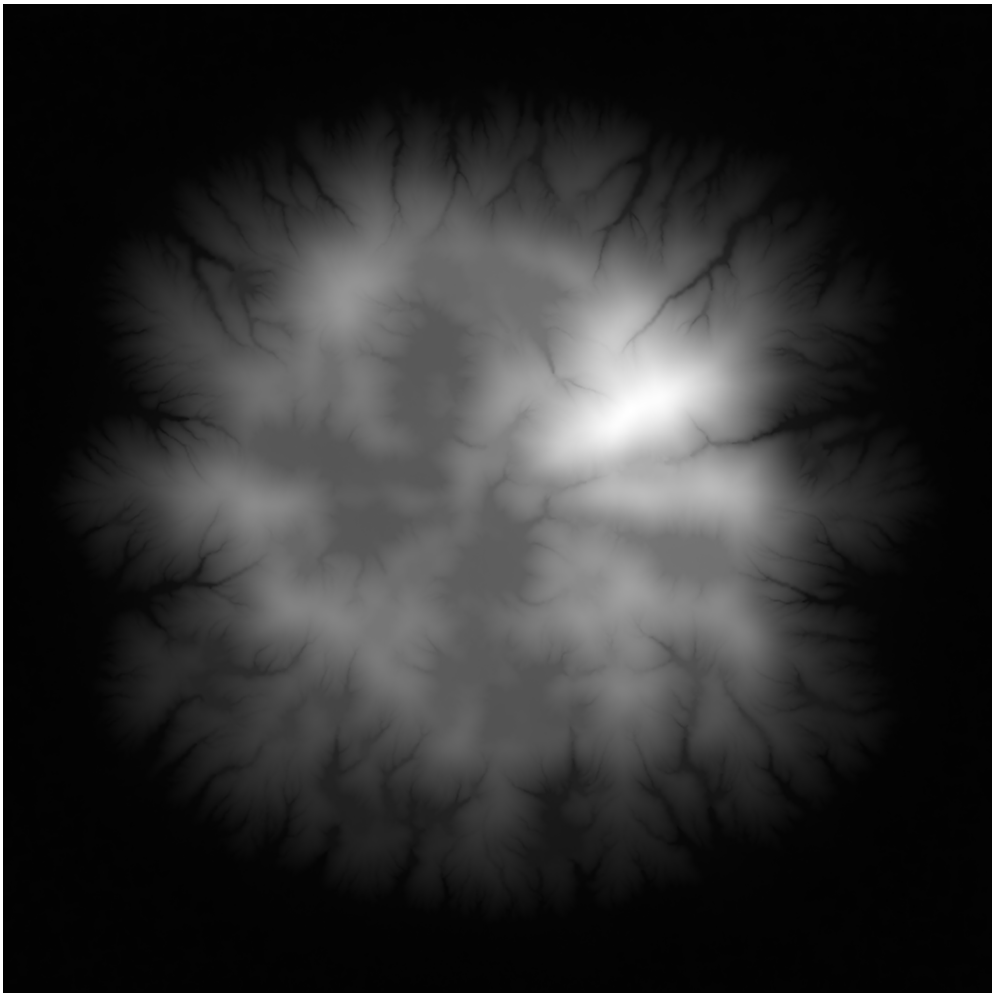
First Stage

This stage calculates the height of the vertices using the height map. The inputs that must be sent to the vertex shader are: the heightmap texture, texture coordinates, the sampler, and the position of the vertex. The texture, texture coordinates and the sampler are passed into the GetHeight function in the Manipulation_Shader.hlsli file, while the vertex position is manipulated separately outside of GetHeight. The GetHeight function gets the colour's x value, this is then used as the height value. To get an accurate height the vertex coordinates must align with the heightmap texture coordinates. Once the height is calculated, but before the vertex position is set to it, it is multiplied by 30, this exaggerates the mountains in the y axis as otherwise they would simply be much too small. Finally back in the vertex shader the vertex y position is then set to this new height value that was multiplied by 30.

Height map after vertex manipulation in wireframe mode:



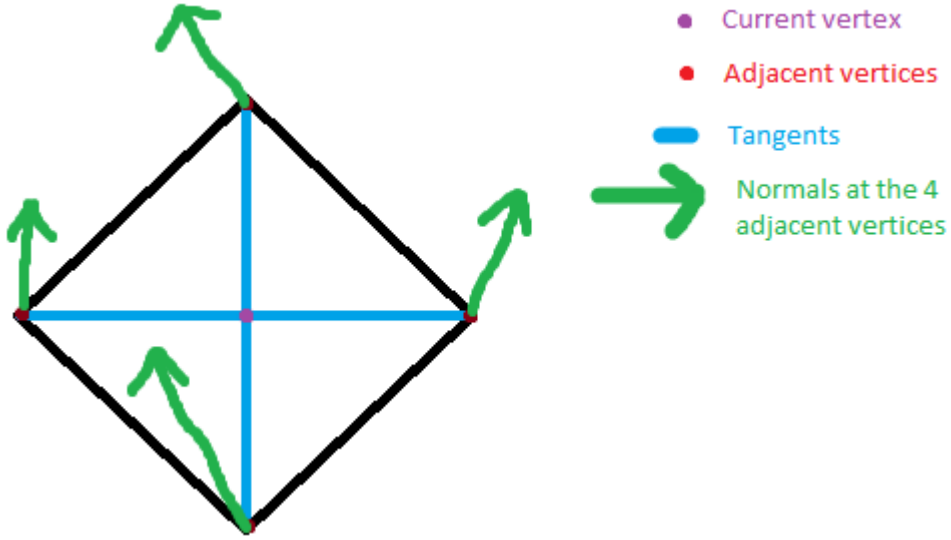
The height map being used



Second Stage

Now the vertices are in the correct location however their normals haven't been calculated. To do this these inputs are required; texture coordinates, the height map texture, the sampler, the mesh quad count, and the same height value from the first stage. A separate function, again in the Manipulation_Shader.hlsli file, called "CalculateVertexNormal" is called. These results will then be passed into the input.normal and will then result in each vertex having correct normals for lighting.

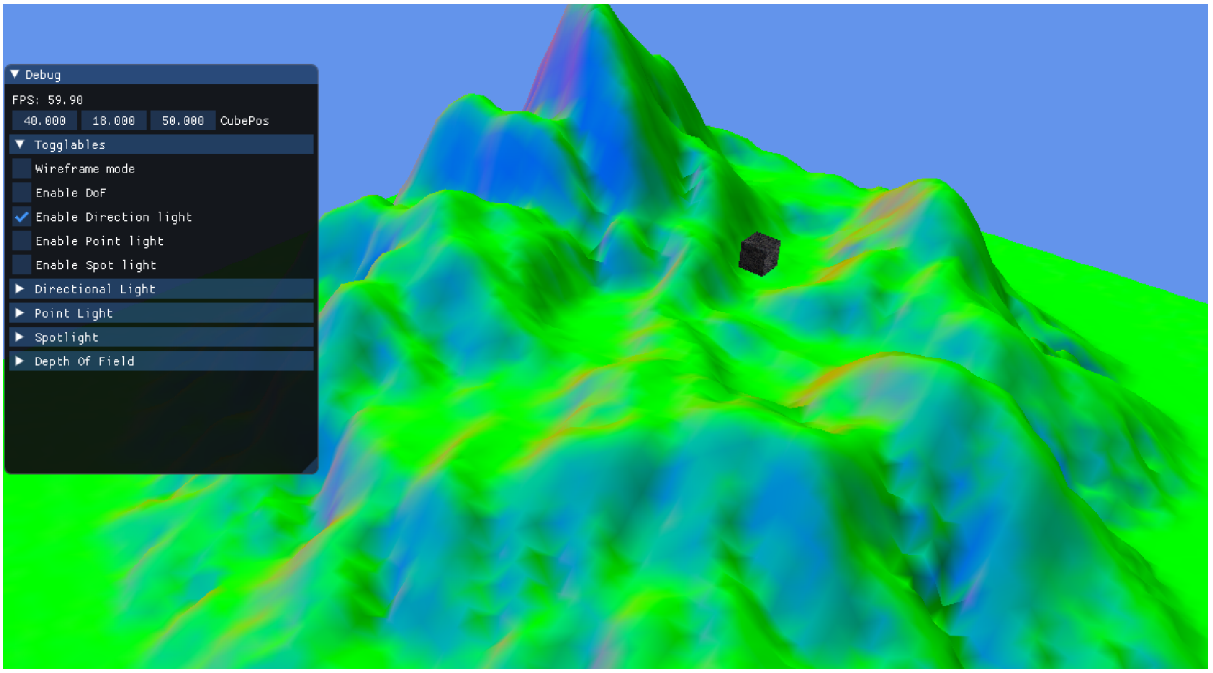
All vertices in the plane are formatted in a grid sequence meaning all but the edge vertices will have a vertex to the east, west, north, and south of it. These can be used to create tangents and calculate the normal of the desired vertex. All of these vertices [N,E,S,W] are equidistant from each other in the plane in the x and z axis, so the interval between them can be calculated using the size of the plane and the mesh quad count.



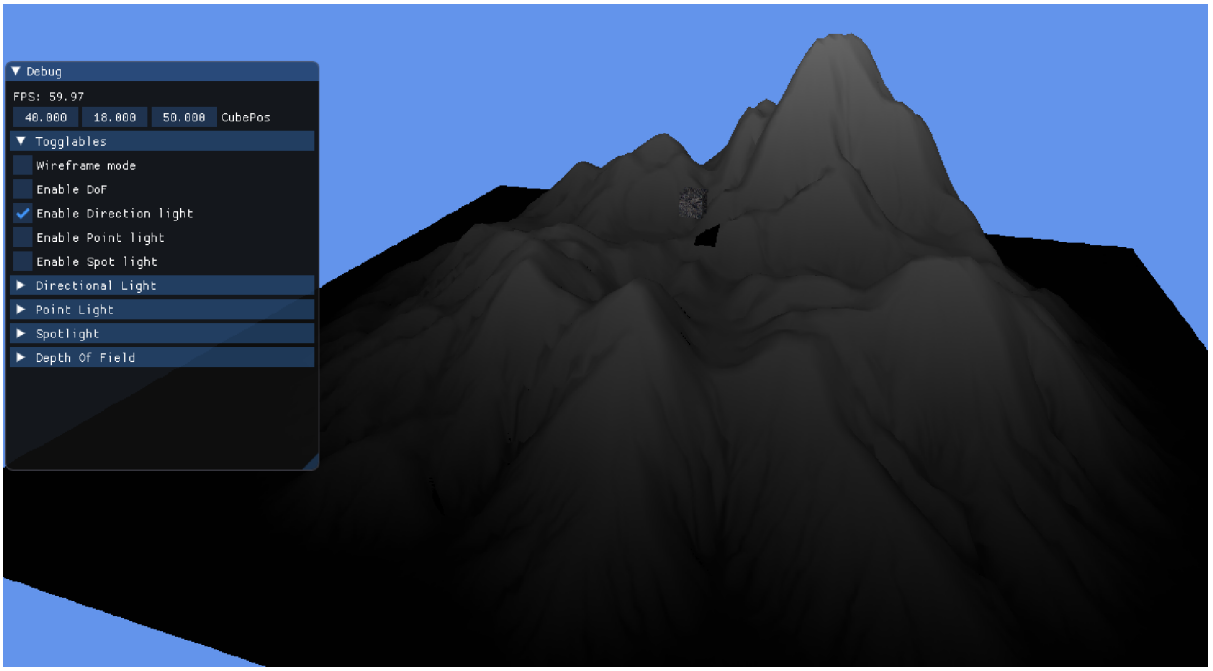
The current vertex and adjacent vertices have their height calculated using the previously mentioned height calculation. A tangent is then calculated at all 4 adjacent vertices . The cross product of these tangents is then calculated (NXE, EXS, SXW, WXN), added together and then divided by 4, this creates an average result, theoretically only 2 tangents are needed however using all 4 gives a more accurate result. Finally use the "normalize" function on this result to equate it to a length of 1 and you have a properly calculated normal.

This process calculates the normals on a per vertex basis meaning the total number of normals is 10000. This is far less than the resolution of the heightmap provides, so if I were to take this further it would create much more accurate lighting doing these calculations per pixel rather than per vertex by utilising bump mapping.

Outputting the calculated normals as RGB values in the manipulation shader



The final output with correct lighting in the manipulation shader

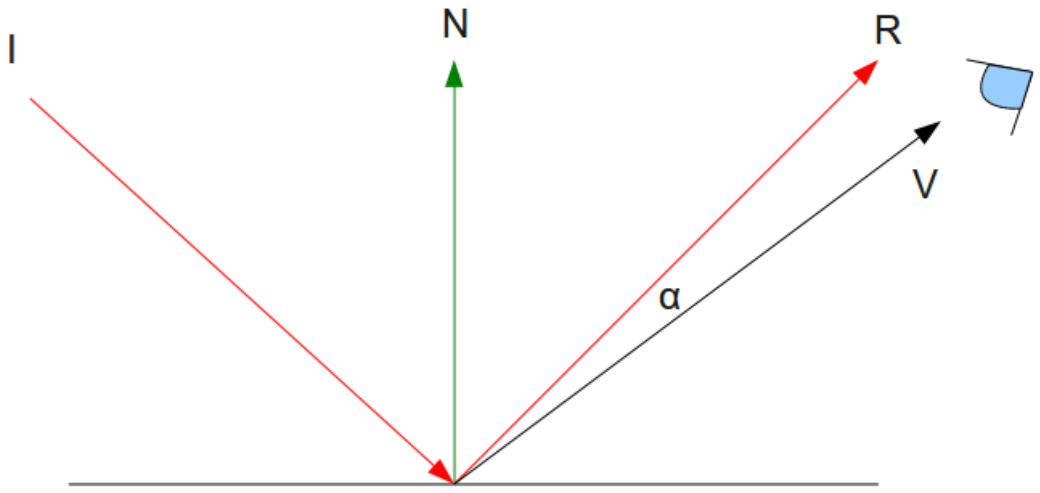


Third Stage - Lighting

As stated in the overview I have 3 different lights within the scene: the directional light which is simply used to light the entire scene and cast shadows, the point light I chose to focus on detail like specular highlights with attenuation, and a spot light which is fully controllable and will cast shadows onto the scene. These lights require multiple sets of inputs and they are all passed to the pixel shader in the light constant buffer. These include the camera position, the direction of the light, the position of the light, the diffuse colour, ambient colour, specular and attenuation values for the point light and the cutoff angle for the spotlight. The status of each light is also passed in this buffer to only calculate a specific light if it is activated.

Specular lighting is a “highlight” effect that applies to the reflection of light on a surface. Practically speaking specular light is when the viewer aligns directly with the reflected ray of light from a surface

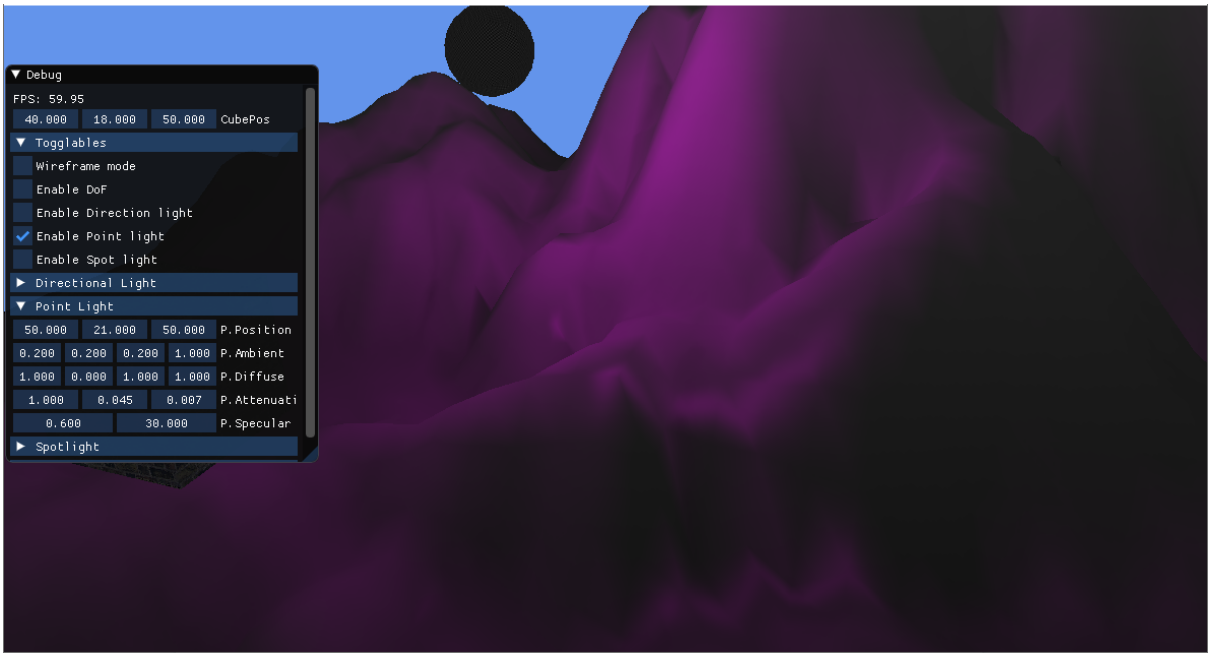
Specular lighting diagram (Hussain)



As shown in the diagram above there is an alpha angle between the viewer and the actual direct reflected ray of light, the closer this angle is to 0 the brighter that light will be, this is most apparent on curved surfaces as only a section of the curve will have this occur so it creates a highlight. To achieve this effect the vector R (from the image above) is calculated and it is compared to the vector from the camera to the pixel using a dot product and is used as a power for the intensity of the light. This would be complete except for the fact that the specular component isn’t attenuated, this would result in the specular highlight being just as bright regardless of how far the camera is from the surface.

Attenuation is the loss of light intensity as distance increases. To calculate this 3 values are needed, the constant attenuation (atten), the linear attenuation (atten1) and quadratic attenuation (atten2), attenuation is then calculated by the formula:
Attenuation = $1/(atten + atten1 * d * atten2 * d^2)$

The specular highlighting being demonstrated by just the point light

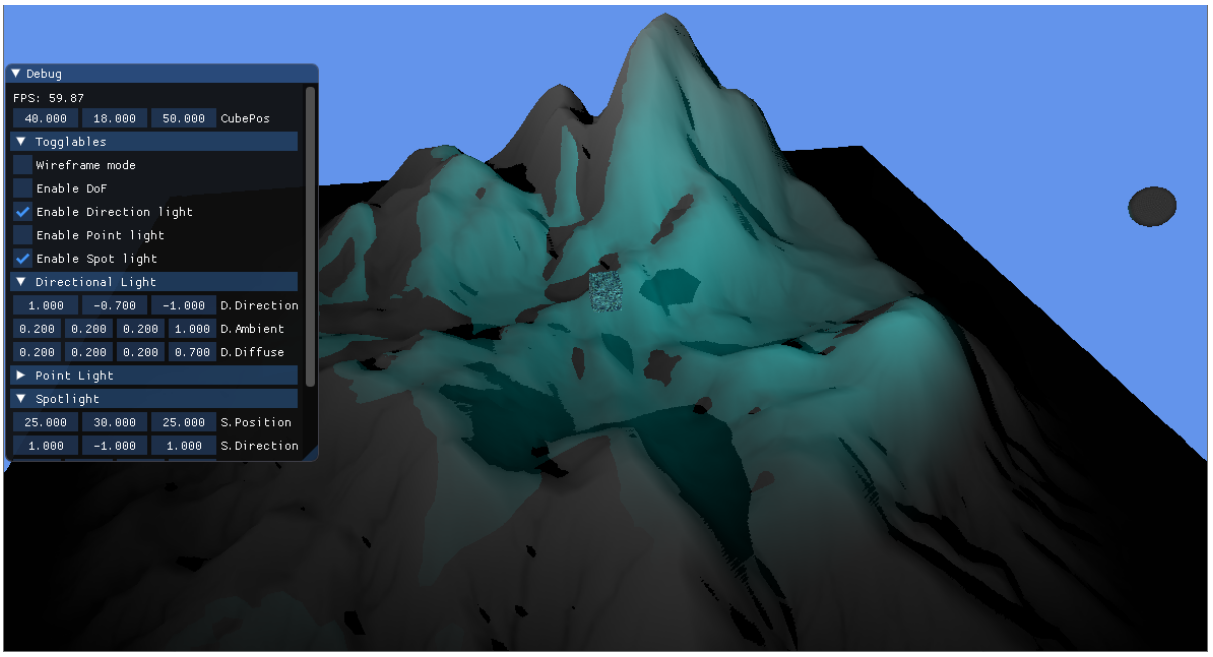


Fourth Stage - Shadows

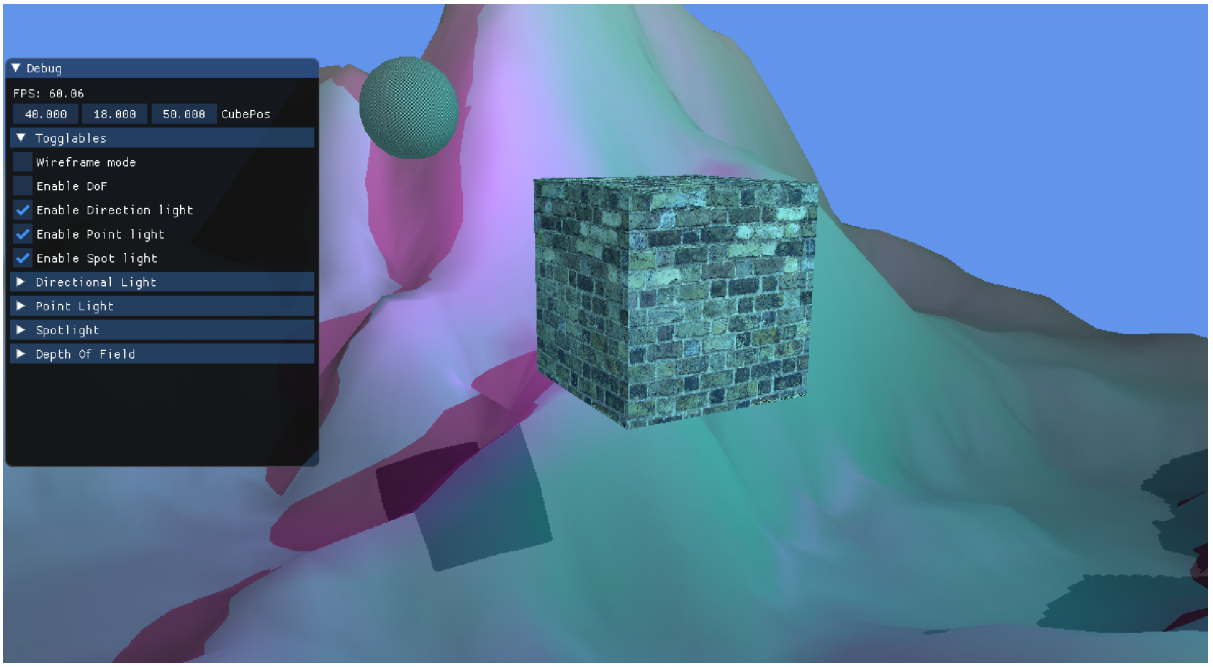
The shadows are calculated and expand upon the existing light functions, I have 2 lights which calculate their shadows, the directional and spot light. The inputs required for shadows are the same as lighting but 2 shadow maps have to be included and 2 light view positions have to be included as well, one for each light.

The light view position for the directional light is multiplied by the world matrix, then further multiplied by the orthographic and view matrices related to the directional light. The light view position for the spotlight is multiplied by the world matrix, and then further multiplied by the perspective and view matrix of the spotlight, the distinction of perspective is because unlike the directional light the spot light doesn't cover the entire scene so it only accounts for what is in range of the spotlight given its current attributes. These calculate which areas of the lights "view" are shadows.

Shadows form both the directional and spot light. The spot light is coloured blue and is in the position of the sphere on the right. This is to clearly identify it from the directional light which colours everything a soft grey.



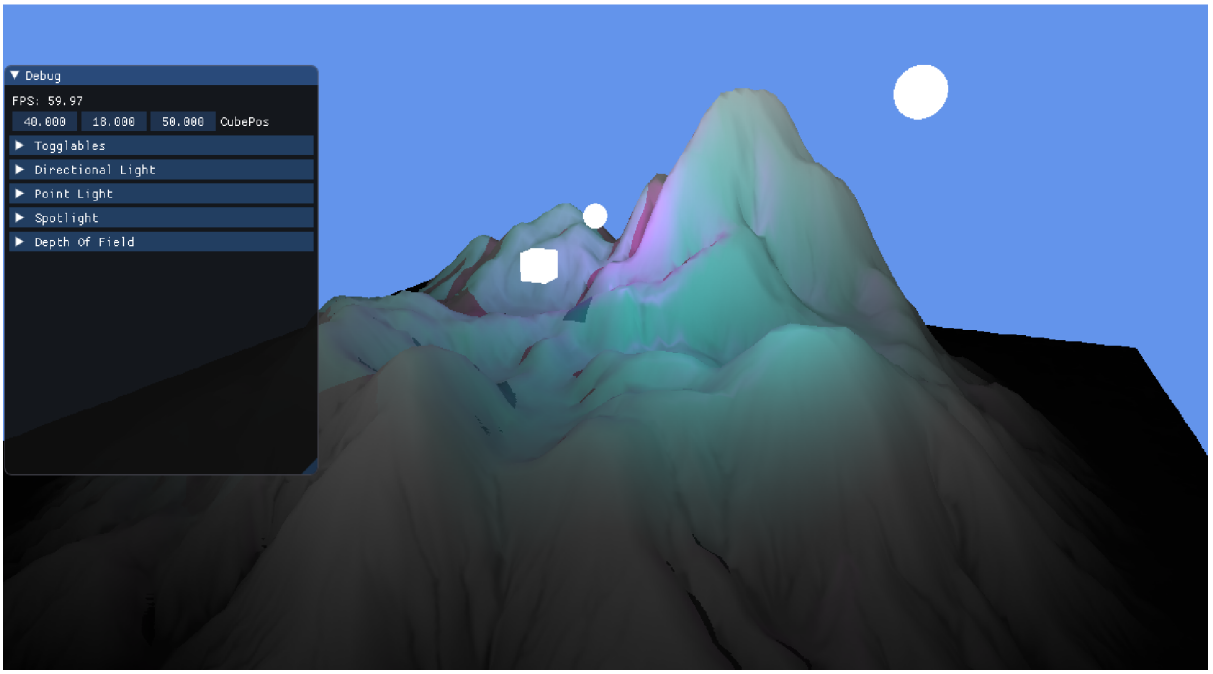
Basic Shader



As shown above there is more geometry in the scene than just the mountains, the shader for said mountains does a lot of manipulation so it is it's own manipulation shader but a more basic shader is still required for the general geometry within the scene. As shown above there is a cube mesh in the map that is used as an extra example for shadows but it still needs to be lit and textured correctly. There are also meshes to show where the point and spot light are within the world, these also need to be rendered in.

This shader simply only handles any basic geometry in the scene, so in my case it is just the light meshes and cube mesh, this could be expanded and just be used for light meshes in a bigger project and a separate shader could be used for generated geometry, however for this case it made sense to group everything together. This shader takes the texture image of the meshes and it runs the same light calculation functions as the manipulation shader does however it never touches the plane mesh, likewise the manipulation shader never touches the general geometry meshes. Once the lighting has been calculated it is multiplied by the relevant texture colour to produce the correct shading on the object.

Here I have set the basic shader to output to white to demonstrate everything it handles



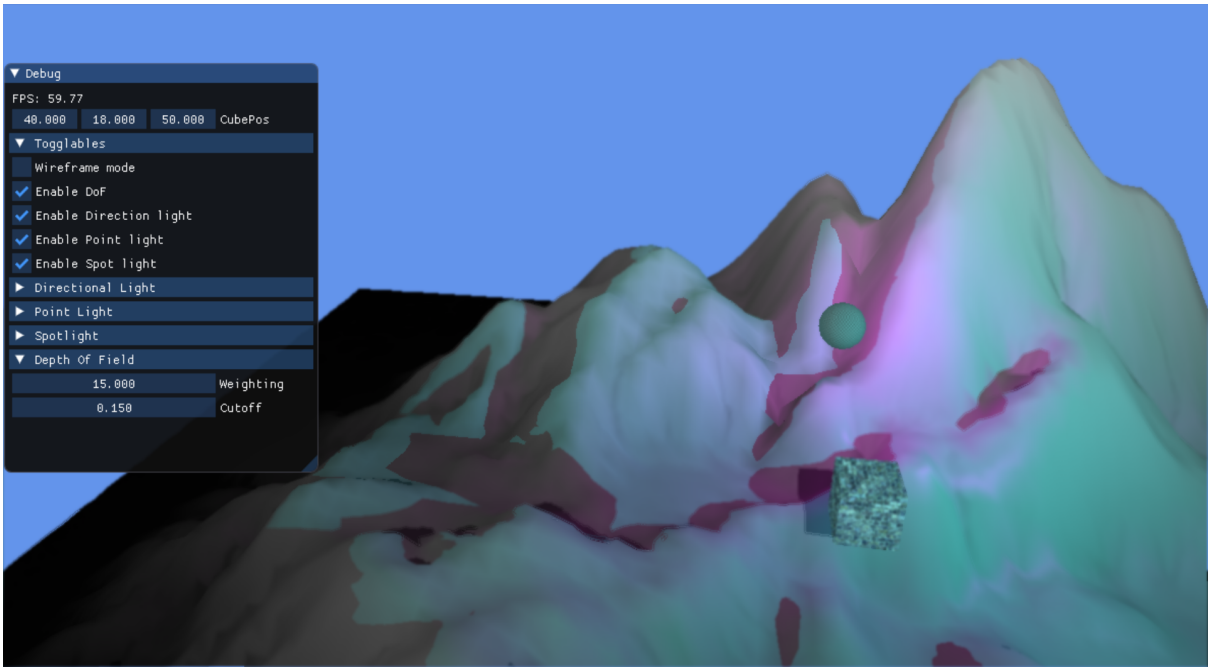
Post-Processing

I have used the post processing effect depth of field. This process takes the depth between the camera and the pixel in the centre of the screen and then blurs the rest of the screen based on its distance to the pixel in the centre of the view. This works for both the mountains and the cube and it allows the user to focus on what they are looking at. Depth of field simulates peripheral vision and focus of the eye.

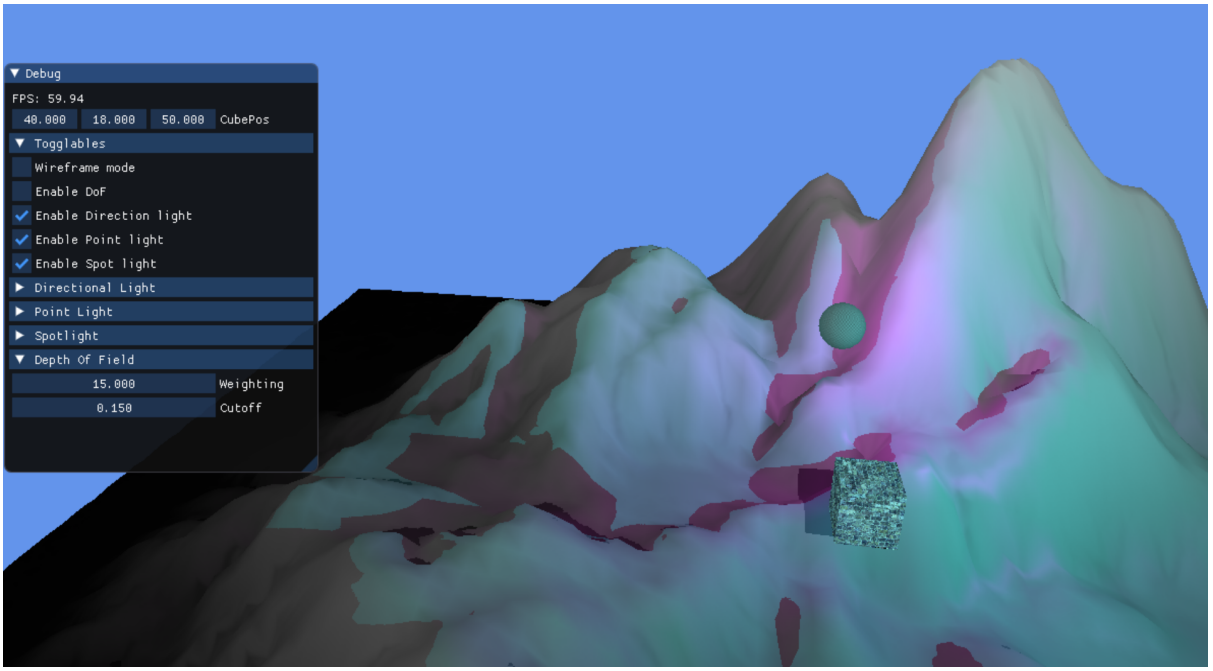
The depth of field calculation takes the depth values and creates a value in the range of 0-1 for the blur on each pixel. This can be made better by adding a cutoff, if a pixel is beyond this cutoff point then its blur is maxed which can dramatise your view a lot but if used subtly creates a smooth transition into a full blur around the edges to fade the image. There is also a weight value added as it allows for a much more customised and harsher blurring of the screen. The weight simply multiplies the blur, amplifying the effect. With a high weight you can get very drastic blurring while with a low weight you can have very subtle blur.

To achieve this effect 5 passes are used; camera depth pass, the screen pass, the horizontal blur pass, the vertical blur pass and the final pass. These passes are used as the order in which these occur is vital and anything being done in the wrong order will break the depth of field as the result from each pass is used in later passes. The final pass is where all the previous pass results are used to finally calculate the depth of field and render it to the screen.

The scene rendered with depth of field enabled



The scene rendered without depth of field enabled



First pass - Camera Depth

The first pass is the camera depth pass. This pass simply returns the depth values based on how far the pixels are from the camera. This pass creates a depth map which is only used again in the final pass. The values you receive from just doing this are incredibly inaccurate when the camera is any real distance away from the pixels. This is due to the result gradient not being linear when it's multiplied through the projection matrix.

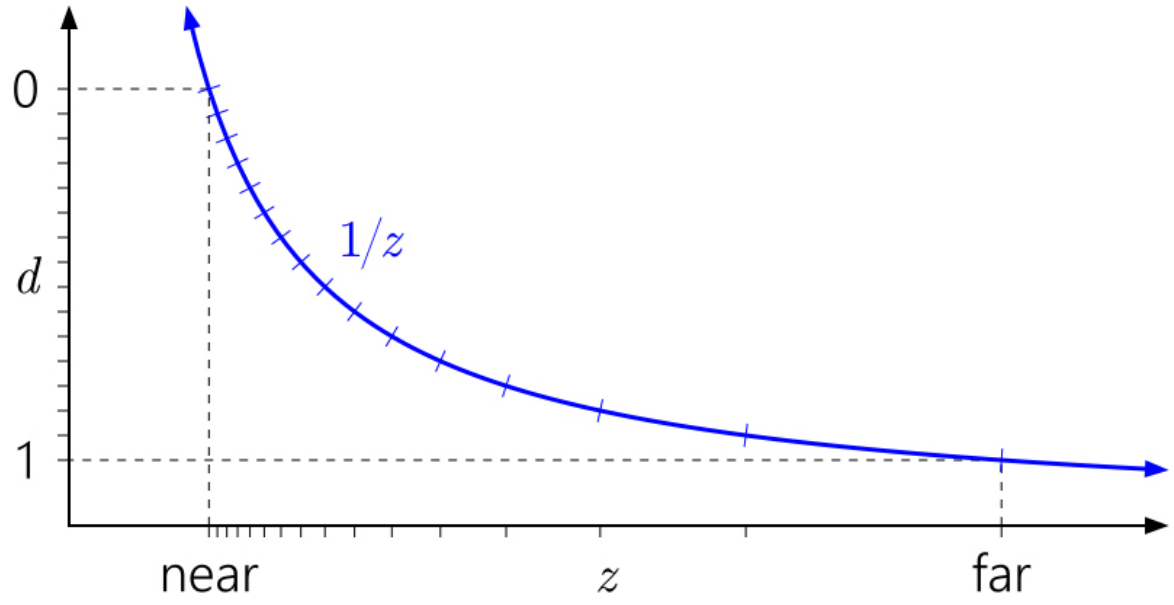
The Z buffer doesn't do much for any long distance calculations of depth and as a result these results are inaccurate and this result gradient must be linearized to give much better results from distance. There is an article on the Nvidia Developer website (2021) that goes into a lot of detail regarding this and multiple solutions but the problem comes down to the fact that the curve for finding the depth map value is reciprocal so all near values are very dense with information but very quickly this drops off and the more deep the pixel is the worse the information spread is.

"I'll use **d** to represent the value stored in the depth buffer (in [0, 1]), and **z** to represent world-space depth, i.e. distance along the view axis, in world units such as meters. In general, the relationship between them is of the form

$$d = a\frac{1}{z} + b$$

where **a,b** are constants related to the near and far plane settings. In other words, **d** is always some linear remapping of **1/z**." (Nvidia, 2021)

Graph depicting the depth map values (Nvidia, 2021)



So to get around this issue a function was used to linearize the depth curve giving consistent and accurate results even for increased depth to pixel values. To achieve this the depth value is multiplied by 2 then subtracted by 1, this gives the same value but in the range of -1

- 1 as opposed to 0 - 1, the inverse of the nonlinear equation is then applied to this value which gets used during the projection matrix.

Nonlinear equation (Learn OpenGL)

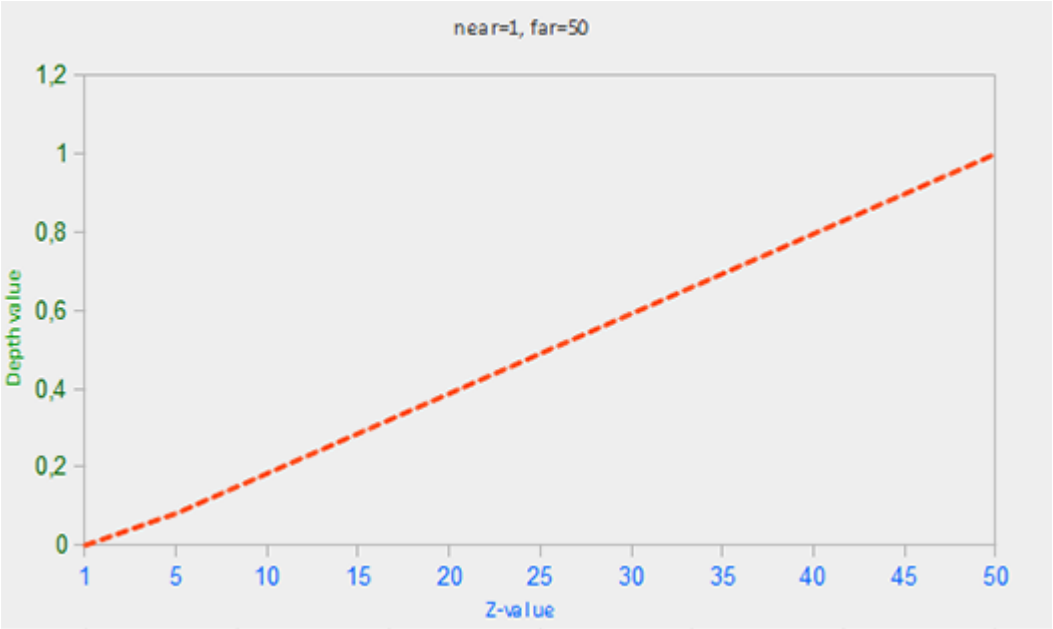
$$F_{depth} = \frac{1/z - 1/near}{1/far - 1/near}$$

Linearized equation (Learn OpenGL)

$$F_{depth} = \frac{z - near}{far - near}$$

This linearized equation is then applied to the calculations for the depth map which gives a much more consistent result regardless of distance, with the sacrifice of losing quality and very close proximities, however since most geometry is being viewed from a distance the benefits would almost never be worth losing the distance quality.

Linearized graph depicting depth map values (Learn OpenGL)



Returning the depth map

Second Pass - Screen Pass

The second pass is the screen pass. This pass renders the entire scene to a render texture. This render texture then gets passed to the next pass (horizontal blur) to be blurred. This pass also gets the information for the lights and locates them in the correct positions and also passes any light calculations.

Third Pass - Horizontal Blur

The horizontal blur pass is the third pass. This pass takes the screen texture from the previous pass and blurs it but only horizontally. This is achieved in the horizontal blur pixel shader using the screen texture, a texture sampler, and the screen width and height. This horizontal blue texture is then sent to the vertical blur pass.

Fourth Pass - Vertical Blur

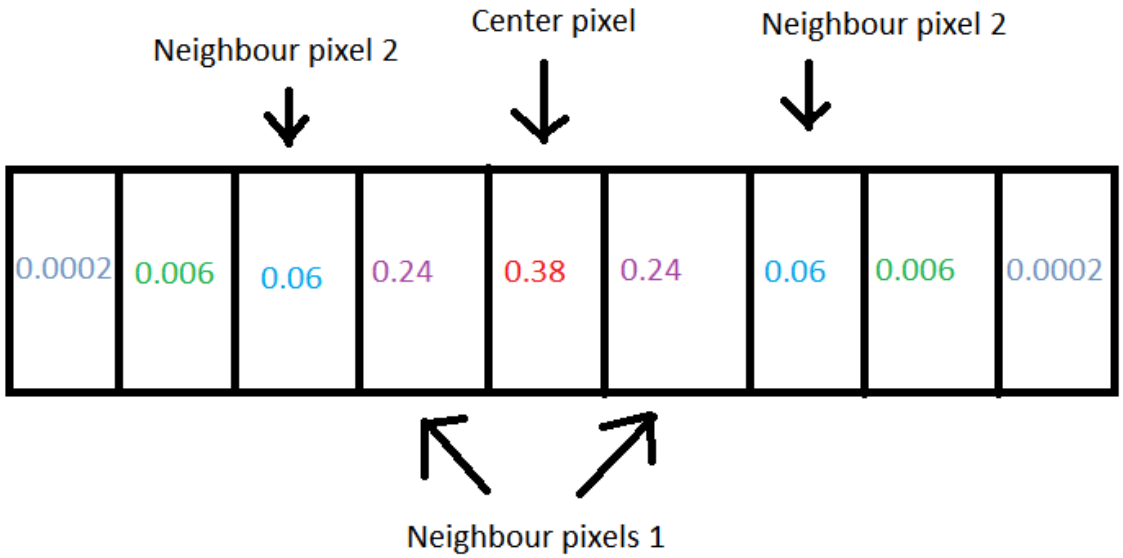
The vertical blur pass is the fourth pass. This pass is very similar to the horizontal blur but it takes the output from the horizontal blue pass and adds vertical blur to complete the total blur. This total blur is then sent to the final pass to add the blur effect of depth of field

Blur Explanation

Both the horizontal and vertical blur are both calculated the same way the only difference being that the horizontal samples pixels horizontally only and the vertical samples vertically only. This explanation is from the perspective of the horizontal blur shader; however, for the vertical shader the same concepts apply; it is just samples vertically instead of horizontally.

The horizontal blur shader takes a sample of 4 neighbouring pixels horizontally. A gaussian blur is then used which involves using a weight system to alter how much a specific pixel will alter the centre pixel. The pixel one to the right of the centre pixel and the pixel one to the left of the centre pixel are both the 1st neighbour and share the same weighting. The weighting gets progressively lower the further from the centre pixel you get, however the total of the weightings will always add up to 1. The quality of the blur will increase as you add more pixels to sample from but for this application 4 neighbours was enough as due to the reduction in weight for each added pixel adding more pixels gives very harsh diminishing returns on the quality..

A diagram to further demonstrate the process of the horizontal blur. Each square is a pixel with the one in red being the centre pixel, the number is the weight



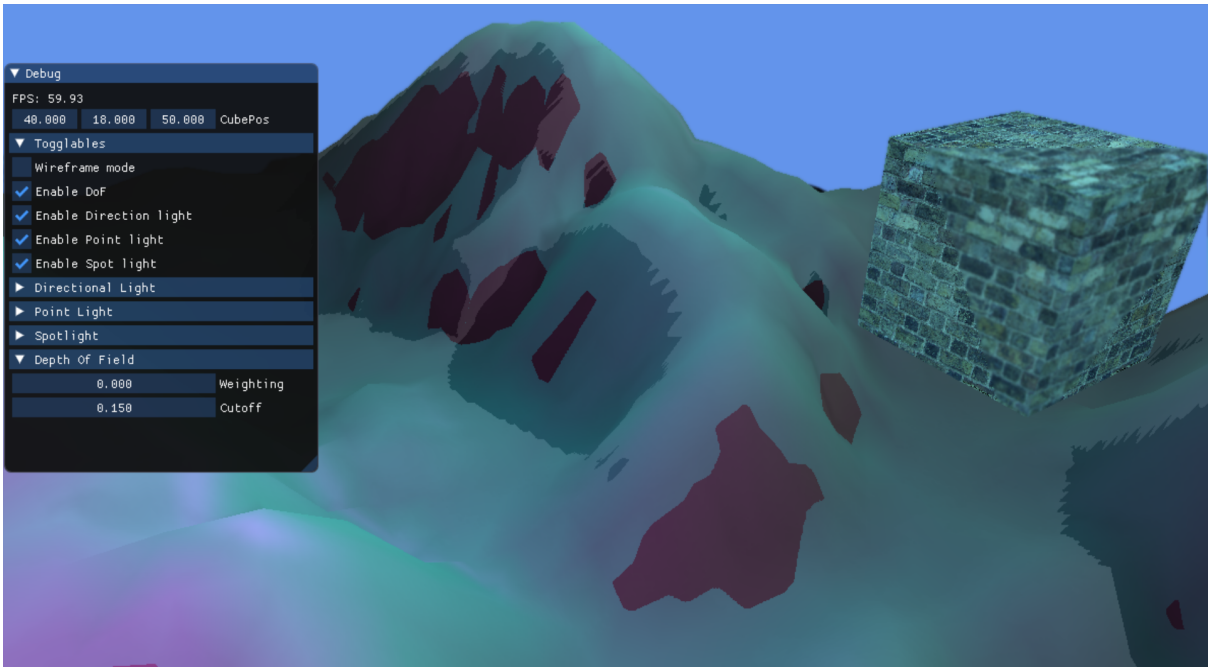
Finally the now horizontally blurred texture is sent to the vertical blur shader, this means that when the vertical blur shader adds it's blur vertically, the horizontal is already done, so the final effect will be as though a 9x9 area of pixels around the centre were sampled as each single pixel in the vertical blur calculation is already sampled, not only the centre pixel.

Fifth Pass - Final Pass

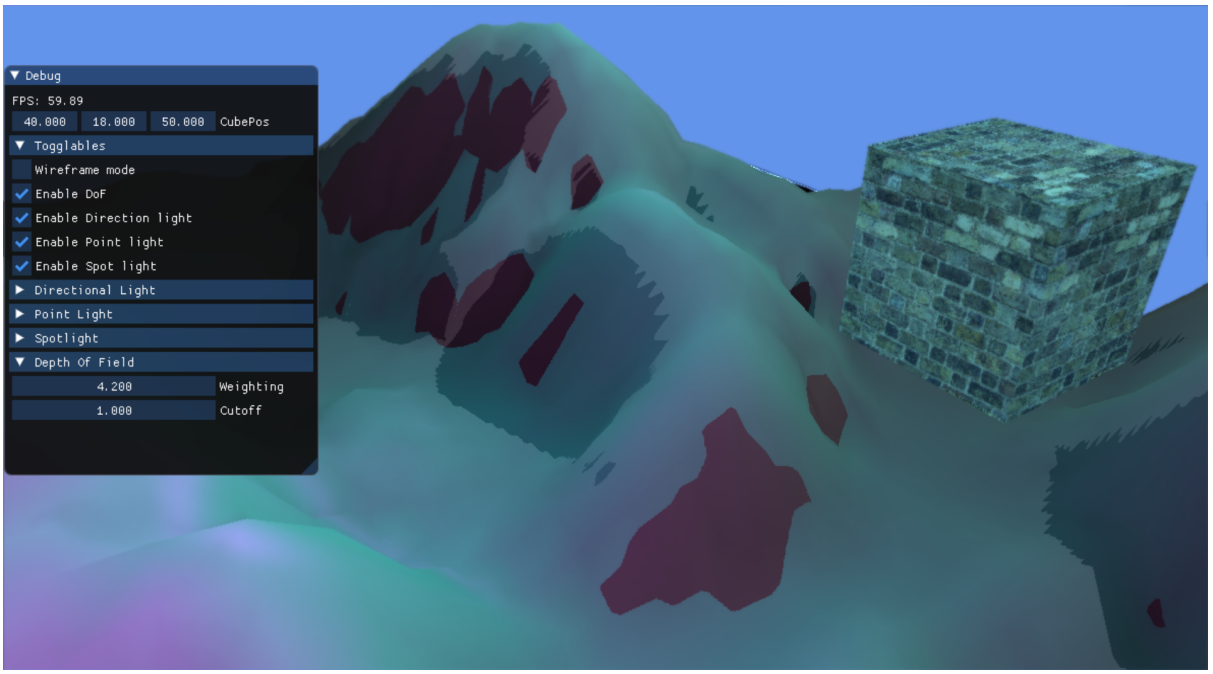
The fifth pass is the final pass. This is the path that takes the returns from all the previous passes to finally put everything together and make the depth of field effect. The depth of field effect is calculated using the 3 final maps that were calculated in previous passes. All of these calculations are done in the depth of field pixel shader. To determine how blurred each pixel is, the shader needs to determine the difference in depth between a pixel and the centre pixel, this process will result in each pixel having a value between 0 - 1. This value is then used as the lerp value between the screen texture (generated in the second pass) and

the blurred texture (generated in the third and fourth pass). This final lerp value can be really small and won't give good results so two solutions were implemented, a cutoff was added which sets the lerp value to 1 if it is above the cutoff threshold, this results in a very harsh transition to full blur so an extra weighting variable was added, this weighting simply multiplies the lerp value to increase its size and give a greater depth of field but rather than a harsh cutoff this is a more gradual method.

The harsh application of depth of field can be seen using the cutoff set to 0.15 here on the cube



The much more smooth transition of blur strength is demonstrated on the cube with a weight of 4.2

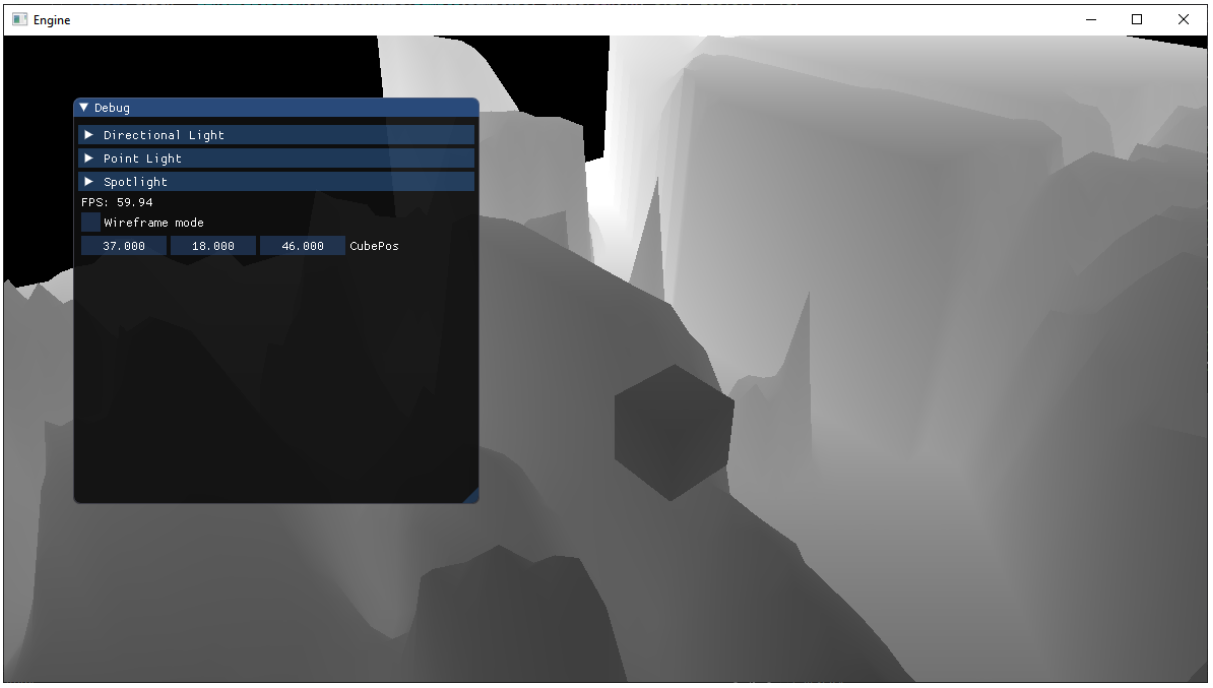


Reflection

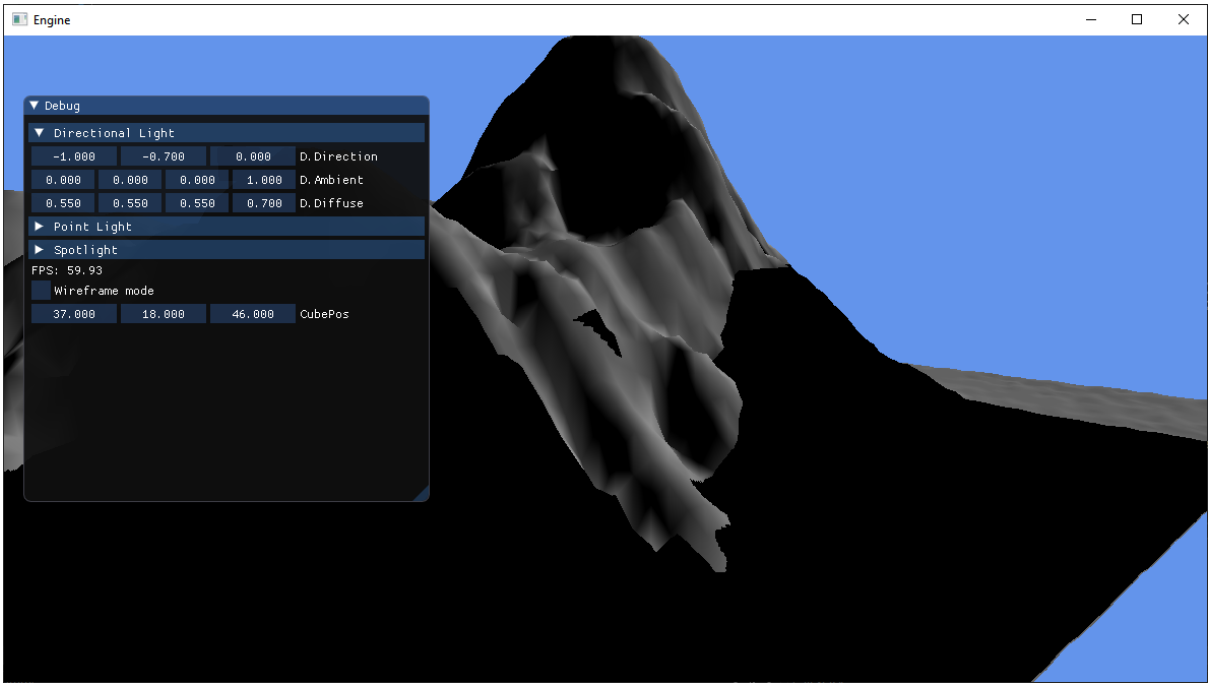
The vertex manipulation is good for what it is however as stated when explaining it, this definitely could have been taken further, exploring options like bump mapping to get per pixel normal calculations. I did start attempting to use a normal map to do a similar thing but due to a lack of time I decided to scrap it and focus on other areas of the project and I knew how many issues I ran into trying to get the per vertex normal calculations working initially.

Before adding the linearize depth function to the depth map calculations I was really unhappy with the results of the field of view but after talking to people and doing research on the depth buffer, I found the linearize depth function and realised how much of a difference it made to the depth buffer.

That being said the depth buffer continued to give me a lot of issues and a common theme for me was making very small mistakes in the code that would take a really long time to debug, once I had linearized the depth my depth map was acting incredibly strange as this was the result I got when I rendered the depth map



This then caused a lot of z-fighting against the manipulated plane mesh in the scene



The mistake I made was in the depth shader cpp file I was passing the same height map texture twice to the same address and once fixing this all the issues disappeared. I still don't know exactly how it caused the issue but making small mistakes like this cost me a lot of time trying to debug them and held me back a lot. I learned with time that programming slowly but carefully ends up significantly faster than trying to rush as trying to find small typos or incorrect field inputs is incredibly difficult especially when running code through multiple shaders and it's not initially clear where the issue lies. This also led to me massively improving my commenting as making everything very clear was the easiest way to avoid making a mistake, some variable names could still be changed such as the plane mesh used for vertex manipulation just being called mesh, but I didn't pay enough attention to it until much later in the project.

The lights I am quite happy with. If I had extra time I would have liked to add another spot light and add shadows to the point light. It would have also been nice to add specular to the spotlights and instead of storing the specular as a property of the light store it as a property of the objects the light was shining on as that way the object would have it's own specular levels and the attenuation on each light would result in a different result.

Other than not calculating point light shadows I think my shadows turned out good, my only issue with them is how harsh they are and it would be nice to learn a way to produce a much softer shadow by perhaps using linear interpolation to increase the shadow as it goes away from its edge.

I believe my UI is very logical and using collapsing headers declutters the screen when you don't need much information on the UI. It would have been nice to have the light toggles in the light headers but I couldn't find a way to have a button toggle the same value in 2 different places.

My example of gaussian blur was really good I think and if I were to make a new project in the future I would definitely keep that and implement it in a new with maybe motion blur as it

would also benefit from separating the horizontal and vertical blur shaders. I was also happy with my lights and I would take them forward but I would make the additions I described above.

I think my application tackles the brief well, it doesn't do a lot to extend itself beyond the basics but I feel that what it does cover is done well, I feel like my point light was done really well with the specular attributes and the scene looks good when just moving single lights around, it would have been really nice to see the specular lighting working on a light with shadow calculations.

References

Hussain, F. (no date) *Specular light diagram*, *OGL dev*. Available at: https://ogldev.org/www/tutorial19/tutorial19.html (Accessed: January 23, 2023).

For depth testing:

Nvidia (2021) *Depth precision visualized*, *NVIDIA Developer*. Available at: https://developer.nvidia.com/content/depth-precision-visualized (Accessed: January 18, 2023).

Learn OpenGL (no date) *Depth testing*, *Learn OpenGL*. Available at: https://learnopengl.com/depth-testing (Accessed: January 18, 2023).